ass6

April 6, 2025

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report,␣
↪confusion_matrix
```

[4]:
```
df = pd.read_csv("C:/Users/cools/Desktop/ML ass/teleCust.csv")
print(df.head())
```

```
   region  tenure  age  marital  address  income  ed  employ  retire  gender  \
0       2      13   44        1        9    64.0   4       5     0.0       0
1       3      11   33        1        7   136.0   5       5     0.0       0
2       3      68   52        1       24   116.0   1      29     0.0       1
3       2      33   33        0       12    33.0   2       0     0.0       1
4       2      23   30        1        9    30.0   1       2     0.0       0

   reside  custcat
0       2        1
1       6        4
2       2        3
3       1        1
4       4        3
```

[6]:
```
# Check for missing values
print(df.isnull().sum())

# Check the data types
print(df.dtypes)

# Convert categorical features if necessary (e.g., 'gender', 'marital')
df['gender'] = df['gender'].astype('category').cat.codes
df['marital'] = df['marital'].astype('category').cat.codes
df['retire'] = df['retire'].astype('category').cat.codes
```

```
region      0
tenure      0
age         0
marital     0
address     0
income      0
ed          0
employ      0
retire      0
gender      0
reside      0
custcat     0
dtype: int64
region       int64
tenure       int64
age          int64
marital      int64
address      int64
income     float64
ed           int64
employ       int64
retire     float64
gender       int64
reside       int64
custcat      int64
dtype: object
```

[8]:
```
# Define feature variables (excluding 'custcat')
X = df.drop(columns=['custcat'])

# Define target variable
y = df['custcat']
```

[10]:
```
# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

# Print dataset shapes
print(f"Training Data Shape: {X_train.shape}")
print(f"Testing Data Shape: {X_test.shape}")
```

```
Training Data Shape: (800, 11)
Testing Data Shape: (200, 11)
```

[12]:
```
# Standardize feature variables
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

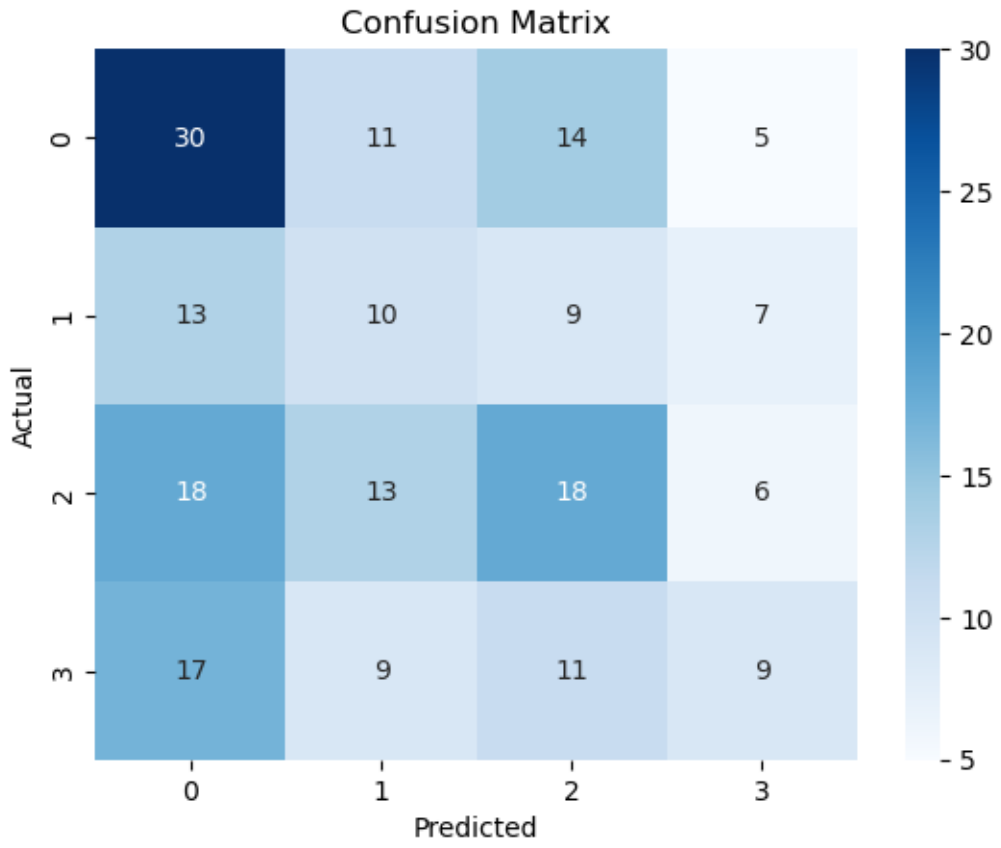
[14]:
```
# Define KNN classifier with k=5
knn = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn.fit(X_train, y_train)
```

[14]:
```
KNeighborsClassifier()
```

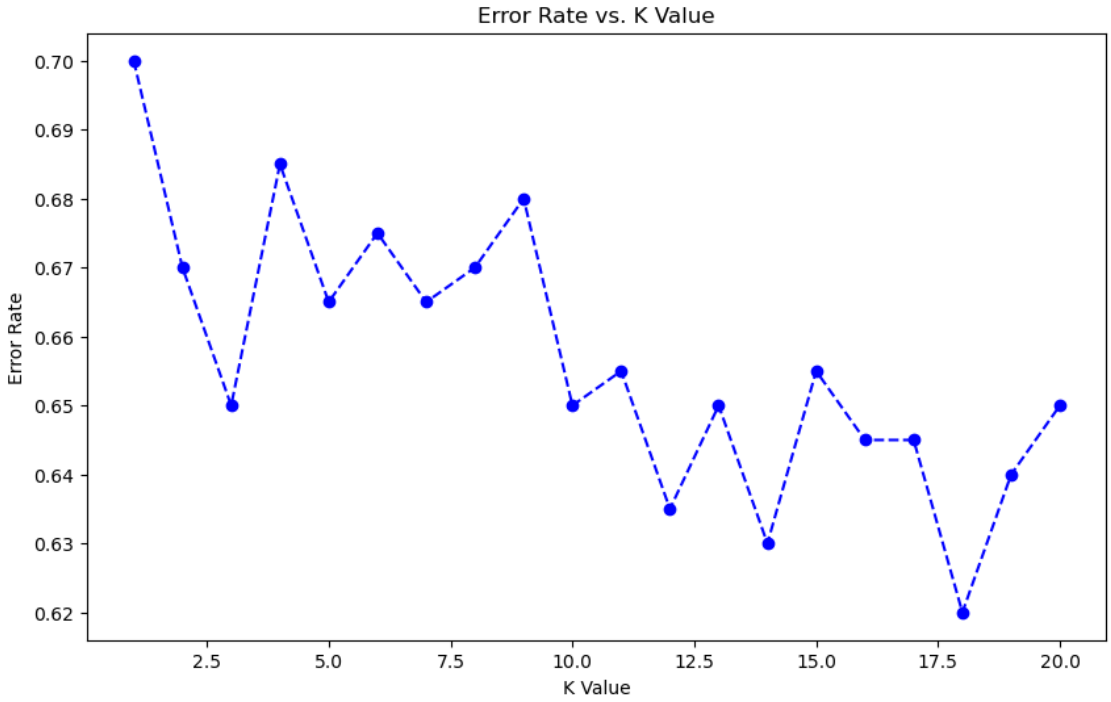
[16]:
```
# Predict on test data
y_pred = knn.predict(X_test)
```

[18]:
```
# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred) * 100
print(f"Accuracy: {accuracy:.2f}%")

# Generate Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
Accuracy: 33.50%

Classification Report:
              precision    recall  f1-score   support

           1       0.38      0.50      0.43        60
           2       0.23      0.26      0.24        39
           3       0.35      0.33      0.34        55
           4       0.33      0.20      0.25        46

    accuracy                           0.34       200
   macro avg       0.32      0.32      0.32       200
weighted avg       0.33      0.34      0.33       200
```



[20]:
```
# Test different k values to find the optimal one
error_rates = []

for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred_k = knn.predict(X_test)
    error_rates.append(1 - accuracy_score(y_test, y_pred_k))

# Plot Error Rate vs K Value
plt.figure(figsize=(10, 6))
plt.plot(range(1, 21), error_rates, marker='o', linestyle='dashed',␣
 ↪color='blue')
plt.xlabel("K Value")
plt.ylabel("Error Rate")
plt.title("Error Rate vs. K Value")
plt.show()
```


Error Rate vs. K Value
Error Rate
K Value
0.70
0.69
0.68
0.67
0.66
0.65
0.64
0.63
0.62
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

[ ]: